

Code File: Data Preprocessing

"""

AI-CliniScan: Data Preprocessing (YOLO Annotation Conversion)

Author: Tirumani Surya

Description: Converts Kaggle bounding box annotations (CSV) into normalized YOLO format (.txt files) for object detection model training.

"""

```
import pandas as pd
import os
import pydicom
from tqdm import tqdm

# Configuration Paths
CSV_PATH = "/kaggle/input/vinbigdata-chest-xray-abnormalities-detection/train.csv"
TRAIN_DIR = "/kaggle/input/vinbigdata-chest-xray-abnormalities-detection/train"
OUTPUT_DIR = "/kaggle/working/processed_train"
LABELS_DIR = "/kaggle/working/labels_train"
os.makedirs(LABELS_DIR, exist_ok=True)

def convert_annotations_to_yolo():
    print("Loading Kaggle CSV...")
    df = pd.read_csv(CSV_PATH)

    processed_pngs = os.listdir(OUTPUT_DIR)
    processed_image_ids = [img.replace('.png', '') for img in processed_pngs]
    df_filtered = df[df['image_id'].isin(processed_image_ids)]

    print(f"Generating YOLO labels for {len(processed_image_ids)} images...")

    for image_id in tqdm(processed_image_ids):
        boxes = df_filtered[df_filtered['image_id'] == image_id]
        label_file_path = os.path.join(LABELS_DIR, f"{image_id}.txt")

        with open(label_file_path, 'w') as f:
            # Handle 'No finding' class (Class ID 14)
            if len(boxes) == 1 and boxes.iloc[0]['class_id'] == 14:
                continue

            # Fetch original image dimensions for normalization
            dicom_path = os.path.join(TRAIN_DIR, f"{image_id}.dicom")
            dicom_data = pydicom.dcmread(dicom_path)
            img_height, img_width = dicom_data.pixel_array.shape

            for _, row in boxes.iterrows():
                if row['class_id'] == 14:
```

```

        continue

    class_id = int(row['class_id'])
    x_min, y_min = row['x_min'], row['y_min']
    x_max, y_max = row['x_max'], row['y_max']

    # YOLO format math (Center X, Center Y, Width, Height)
    bbox_width = x_max - x_min
    bbox_height = y_max - y_min
    x_center = x_min + (bbox_width / 2)
    y_center = y_min + (bbox_height / 2)

    # Normalize coordinates
    x_center /= img_width
    y_center /= img_height
    bbox_width /= img_width
    bbox_height /= img_height

    f.write(f"{class_id} {x_center:.6f} {y_center:.6f} {bbox_width:.6f}
{bbox_height:.6f}\n")

print("YOLO Annotation conversion complete.")

if __name__ == "__main__":
    convert_annotations_to_yolo()

```